Ejercicio 1

```
def lee_rutas(fichero):
    rutas = []
    with open(fichero) as f:
        lector = csv.reader(f, delimiter = ";")
        next(lector)
        for ciudad_inicio, coordenada, fecha_ruta, km, gasolineras, dificultad,\
             zona_descanso, vel_max, vel_min in lector:
            fecha_ruta = parsea_fecha(fecha_ruta)
            km = float(km)
            gasolineras = int(gasolineras)
            zona_descanso = parsea_booleano(zona_descanso)
            vel_max = int(vel_max)
            vel_min = int(vel_min)
            rutas.append(Ruta(ciudad_inicio.strip(), parsea_coordenada(coordenada),
            fecha_ruta, km, gasolineras, dificultad, zona_descanso, vel_max, vel_min))
    return rutas

def parsea_fecha(cadena):
    if cadena != "":
        res= datetime.strptime(cadena, "%m/%d/%Y").date()
    else:
        res = datetime.today().date()
    return res

def parsea_booleano(cadena):
    res = None
    if cadena == "True":
        res = True
    else:
        res = False
    return res

def parsea_coordenada(cadena):
    coordenada = cadena.split("/")
    return Coordenada(float(coordenada[0].strip()), float(coordenada[1].strip()))
```

Módulo rutas_test

```
def mostrar_iterable(iterable):
    for i in range(len(iterable)):
        print(f"{i} - {iterable[i]}")

def mostrar_diccionario(dic):
    for c,v in dic.items():
        print(f"{c} - {v}")

def test_lee_rutas(rutas):
```

```
    print(f"Se han leido {len(rutas)} rutas")
    print(f"Los tres primeros registros son:")
    mostrar_iterable(rutas[:3])
    print(f"Los tres últimos registros son:")
    mostrar_iterable(rutas[-3:])

if __name__== "__main__":
    rutas = lee_rutas('../data/rutas_motos.csv')
    test_lee_rutas(rutas)
```

Ejercicio 2

```
def acumular_kms_por_meses(rutas):
    d= agrupar_por_anyos(rutas)
    res = dict()
    for anyo, rutas_anyo in d.items():
        d_total_meses = acumular_por_meses(rutas_anyo)
        total_meses_anyo = [d_total_meses.get(mes, 0) for mes in range(1, 13)]
        res[anyo] = total_meses_anyo
    return res

def agrupar_por_anyos(rutas):
    res = dict()
    for ruta in rutas:
        clave = ruta.fecha_ruta.year
        if clave in res:
            res[clave].append(ruta)
        else:
            res[clave] = [ruta]
    return res

def acumular_por_meses(rutas):
    res = dict()
    for ruta in rutas:
        clave = ruta.fecha_ruta.month
        if clave in res:
            res[clave] += ruta.km
        else:
            res[clave] = ruta.km
    return res
```

Módulo rutas_test

```
def test_acumular_kms_por_meses(rutas):
    print(">>  TEST DE LA FUNCIÓN acumular_kms_por_meses:  ")
    resultado = acumular_kms_por_meses(rutas)
    mostrar_diccionario(resultado)
```

```
if __name__== "__main__":
    rutas = lee_rutas('../data/rutas_motos.csv')
    test_acumular_kms_por_meses(rutas)
```

Ejercicio 3

```
def diferencias_kms_meses_anyo(rutas):
    d = acumular_kms_por_meses(rutas)
    return {anyo: incrementos_kms(kms_meses) \
            for anyo, kms_meses in d.items()}

def incrementos_kms(total_meses):
    res = []
    for i in range(len(total_meses) - 1):
        res.append(total_meses[i+1] - total_meses[i])
    return res
```

Soluciones alternativas para la función incrementos_kms

```
def incrementos_kms(total_meses):
    res = [total_meses[i+1] - total_meses[i] for i in range(len(total_meses) - 1)]

def incrementos_kms(total_meses):
    res = [total2 - total1 for total1, total2 in zip(total_meses, total_meses[1:])]

def incrementos_kms(total_meses):
    res = []
    for total1, total2 in zip(total_meses, total_meses[1:]):
        res.append(total2 - total1)
    return res
```

Solución alternativa

```
def diferencias_kms_meses_anyo(rutas):
    res = dict()
    r_aux = acumular_kms_por_meses(rutas)

    for anyo in r_aux:
        l_aux = []
        for i in range(len(r_aux[anyo]) - 1):
            l_aux.append(r_aux[anyo][i+1] - r_aux[anyo][i])
        res[anyo] = l_aux
    return res
```

Módulo rutas_test

```
def test_diferencias_kms_meses_anyo(rutas):
    print(">>  TEST DE LA FUNCIÓN diferencias_kms_meses_anyo:  ")
```

```
    resultado = diferencias_kms_meses_anyo(rutas)
    mostrar_diccionario(resultado)

if __name__== "__main__":
    RUTAS = lee_rutas('./data/rutas_motos.csv')
    test_diferencias_kms_meses_anyo(RUTAS)
```

Ejercicio 4

```
def top_rutas_lejanas(rutas, n,coordenada, km_min=None):
    return sorted ((ruta for ruta in rutas if km_min == None or ruta.km >= km_min),\
                key = lambda ruta: distancia_manhattan(ruta.coordenada, coordenada), \
                reverse = True)[:n]
```

Solución alternativa

```
def top_rutas_lejanas(rutas, n, coordenada, km_min=None):
    if km_min != None:
        rutas = [r for r in rutas if r.km >= km_min]
    rutas_ordenadas = sorted(rutas, \
        key=lambda r: distancia_manhattan(r.coordenada, coordenada), reverse = True)
    return rutas_ordenadas[:n]
```

Módulo coordenadas

```
def distancia_manhattan (coordenada1, coordenada2):
    return abs(coordenada2.latitud - coordenada1.latitud) + \
           abs(coordenada2.longitud - coordenada1.longitud)
```

Módulo rutas_test

```
def test_top_rutas_lejanas(rutas, n, coordenada, km_min=None):
    print(">>  TEST DE LA FUNCIÓN top_rutas_lejanas:  ")
    print (f"Las {n} rutas más cercanas a {coordenada} con más de {km_min} km son")
    res = top_rutas_lejanas(rutas, n, coordenada, km_min)
    mostrar_iterable(res)

if __name__== "__main__":
    rutas = lee_rutas('../data/rutas_motos.csv')
    test_top_rutas_lejanas(rutas, 2, Coordenada(35.15, -8.76), km_min = None)
```

Ejercicio 5

```
def ciudades_top_tiempo_dificultad(rutas,  n=3):
    d = agrupar_rutas_con_zona_descanso_por_dificultad(rutas)
    return {dificultad: ciudades_top_por_tiempo(lista_rutas, n) \
                 for dificultad, lista_rutas in d.items()}
```

```
def agrupar_rutas_con_zona_descanso_por_dificultad(rutas):
    res = dict()
    for ruta in rutas:
        if ruta.zona_descanso:
            clave = ruta.dificultad
            if clave in res:
                res[clave].append(ruta)
            else:
                res[clave] = [ruta]
    return res

def ciudades_top_por_tiempo(rutas, n):
    r_ord= sorted((ruta for ruta in rutas), key = tiempo)[:n]
    return [ruta.ciudad_inicio for ruta in r_ord]

def tiempo (ruta):
    return ruta.km / ruta.vel_min
```

Solución alternativa

```
def agrupa_por_dificultad(rutas):
    res = dict()
    for r in rutas:
        clave = r.dificultad
        if clave in res:
            res[clave].append(r)
        else:
            res[clave] = [r]
    return res

def ciudades_top_tiempo_dificultad2(rutas, n = 3):
    rutas = [r for r in rutas if r.zona_descanso]
    res = agrupa_por_dificultad(rutas)
    for dificultad, tuplas in res.items():
        lista = sorted(tuplas, key = lambda p: p.km/p.vel_min)[:n]
        res[dificultad] = [l.ciudad_inicio for l in lista]
    return res
```

Módulo rutas_test

```
def test_ciudades_top_tiempo_dificultad(rutas):
    print(">>  TEST DE LA FUNCIÓN ciudades_top_tiempo_dificultad:    ")
    resultado = ciudades_top_tiempo_dificultad(rutas)
    mostrar_diccionario(resultado)

if __name__== "__main__":
    rutas = lee_rutas('../data/rutas_motos.csv')
    test_ciudades_top_tiempo_dificultad(rutas)
```